

EECS 348 – Assignment 2: GenAI Analysis

Name: Cesar Olono Munoz

GenAI Programs Used:

1. ChatGPT
2. Microsoft Copilot

How Each Was Accessed:

ChatGPT — accessed via chatgpt.com in browser

Copilot — accessed via copilot.microsoft.com in browser

Prompt Used For Both:

Write a C program that manages a priority queue of emails for a busy company CEO's inbox, using a MaxHeap implemented from scratch with a list-based (linked list) implementation — do not use any pre-existing heap library or module.

The program reads commands from a text file, one per line, in these formats:

EMAIL <sender category>,<subject line>,<date> — adds an email to the inbox. <sender category> is one of: Boss, Subordinate, Peer, ImportantPerson, OtherPerson (in that priority order, Boss highest). <subject line> is a string that may contain spaces but not commas. <date> is in MM-DD-YYYY format. If multiple emails share the same sender category, the newest date should have higher priority (be read first).

NEXT — display the next email to be read, in this format:

Next email:

Sender: <category>

Subject: <subject>

Date: <date>

without removing it from the queue.

READ — remove the current highest-priority email from the queue (whether or not NEXT was called on it first) without displaying anything.

COUNT — display the current number of unread emails in this format: There are N emails to read.

Handle edge cases: NEXT or READ when the queue is empty; two NEXTs in a row without a READ between them (should show the same email both times); two READs in a row without a NEXT between them (the second READ silently removes the next-highest email without displaying it).

Provide the complete C source code.

d. Generated Code from Both GenAIs

Chatgpt's:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <ctype.h>
```

```
#define MAX_LINE 1024
```

```
#define MAX_SUBJECT 512
```

```
#define MAX_CATEGORY 32
```

```
#define MAX_DATE 11
```

```
/* -----
```

```
    Email structure
```

```
----- */
```

```
typedef struct Email {
```

```
    char category[MAX_CATEGORY];
```

```
    char subject[MAX_SUBJECT];
```

```
    char date[MAX_DATE];
```

```
    int categoryPriority;
```

```
int dateValue;
```

```
/* Position in the linked-list representation of the heap */
```

```
int heapIndex;
```

```
} Email;
```

```
/* -----
```

```
Linked-list node
```

```
----- */
```

```
typedef struct Node {
```

```
    Email *email;
```

```
    struct Node *next;
```

```
} Node;
```

```
/* -----
```

```
MaxHeap implemented using a linked list.
```

The linked list stores heap elements in level-order:

index 0 = root

index 1 = left child of root

index 2 = right child of root

index 3 = left child of index 1

...

Parent of index i:

$$(i - 1) / 2$$

Left child:

$$2*i + 1$$

Right child:

$$2*i + 2$$

----- */

typedef struct {

*Node *head;*

int size;

} MaxHeap;

/ -----*

Get sender-category priority.

Higher number = higher priority.

----- */

*int getCategoryPriority(const char *category)*

{

if (strcmp(category, "Boss") == 0)

return 5;

if (strcmp(category, "Subordinate") == 0)

return 4;

```

    if (strcmp(category, "Peer") == 0)
        return 3;

    if (strcmp(category, "ImportantPerson") == 0)
        return 2;

    if (strcmp(category, "OtherPerson") == 0)
        return 1;

    return 0;
}

/* -----
   Convert MM-DD-YYYY into an integer that preserves
   chronological ordering.

   YYYYMMDD is used, so a larger value means a newer date.
   ----- */

int convertDate(const char *date)
{
    int month, day, year;

    if (sscanf(date, "%d-%d-%d", &month, &day, &year) != 3)
        return 0;

    return year * 10000 + month * 100 + day;
}

```

```
}
```

```
/* -----
```

Compare two emails.

Returns:

> 0 if a has higher priority than b

< 0 if a has lower priority than b

= 0 if they have equal priority

Sender category is considered first.

If categories are equal, newer date wins.

```
----- */
```

```
int compareEmails(const Email *a, const Email *b)
```

```
{
```

```
    if (a->categoryPriority != b->categoryPriority)
```

```
        return a->categoryPriority - b->categoryPriority;
```

```
    if (a->dateValue != b->dateValue)
```

```
        return a->dateValue - b->dateValue;
```

```
    return 0;
```

```
}
```

```
/* -----
```

Find the linked-list node at a zero-based heap index.

This is $O(n)$, which is expected because the heap is explicitly implemented using a linked list.

```
----- */
```

```
Node *getNodeAt(MaxHeap *heap, int index)
```

```
{
```

```
    Node *current;
```

```
    int i;
```

```
    if (index < 0 || index >= heap->size)
```

```
        return NULL;
```

```
    current = heap->head;
```

```
    for (i = 0; i < index && current != NULL; i++)
```

```
        current = current->next;
```

```
    return current;
```

```
}
```

```
/* -----
```

Swap two heap elements.

```

----- */

void swapElements(MaxHeap *heap, int index1, int index2)
{
    Node *node1 = getNodeAt(heap, index1);
    Node *node2 = getNodeAt(heap, index2);
    Email *temp;

    if (node1 == NULL || node2 == NULL)
        return;

    temp = node1->email;
    node1->email = node2->email;
    node2->email = temp;
}

/* -----
   Move an element upward until the max-heap property holds.
   ----- */

void heapifyUp(MaxHeap *heap, int index)
{
    int parentIndex;
    Node *current;
    Node *parent;

```

```

while (index > 0) {
    parentIndex = (index - 1) / 2;

    current = getNodeAt(heap, index);
    parent = getNodeAt(heap, parentIndex);

    if (current == NULL || parent == NULL)
        break;

    if (compareEmails(current->email, parent->email) <= 0)
        break;

    swapElements(heap, index, parentIndex);

    index = parentIndex;
}
}

/* -----
Move an element downward until the max-heap property holds.
----- */

void heapifyDown(MaxHeap *heap, int index)
{
    int left, right, largest;

```

*Node *current;*

*Node *leftNode;*

*Node *rightNode;*

while (1) {

*left = 2 * index + 1;*

*right = 2 * index + 2;*

largest = index;

current = getNodeAt(heap, index);

if (current == NULL)

break;

if (left < heap->size) {

leftNode = getNodeAt(heap, left);

if (leftNode != NULL &&

compareEmails(leftNode->email, current->email) > 0) {

largest = left;

}

}

if (right < heap->size) {

```
Node *largestNode = getNodeAt(heap, largest);
```

```
rightNode = getNodeAt(heap, right);
```

```
if (rightNode != NULL &&
```

```
    largestNode != NULL &&
```

```
    compareEmails(rightNode->email, largestNode->email) > 0) {
```

```
    largest = right;
```

```
}
```

```
}
```

```
if (largest == index)
```

```
    break;
```

```
swapElements(heap, index, largest);
```

```
index = largest;
```

```
}
```

```
}
```

```
/* -----
```

```
Initialize an empty heap.
```

```
----- */
```

```
void initHeap(MaxHeap *heap)
```

```
{
```

```
    heap->head = NULL;
```

```

    heap->size = 0;
}

/* -----
   Insert an email into the max heap.
   ----- */
void insertHeap(MaxHeap *heap, Email *email)
{
    Node *newNode;
    Node *last;

    newNode = malloc(sizeof(Node));

    if (newNode == NULL) {
        fprintf(stderr, "Memory allocation failed.\n");
        exit(EXIT_FAILURE);
    }

    newNode->email = email;
    newNode->next = NULL;

    /*
       * Append to the end of the linked list.
       * The resulting list remains in heap level-order.
    */

```

```

    */

    if (heap->head == NULL) {

        heap->head = newNode;

    } else {

        last = getNodeAt(heap, heap->size - 1);

        last->next = newNode;

    }

    heap->size++;

    /* Restore max-heap property. */
    heapifyUp(heap, heap->size - 1);
}

/* -----
Remove and return the highest-priority email.

Returns NULL if the heap is empty.
----- */

Email *removeMax(MaxHeap *heap)
{
    Node *rootNode;

    Node *lastNode;

    Node *previous;

```

```
Email *result;
```

```
if (heap->size == 0)
```

```
    return NULL;
```

```
rootNode = heap->head;
```

```
result = rootNode->email;
```

```
/*
```

```
 * If there is only one element, simply remove the root.
```

```
*/
```

```
if (heap->size == 1) {
```

```
    heap->head = NULL;
```

```
    heap->size = 0;
```

```
    free(rootNode);
```

```
    return result;
```

```
}
```

```
/*
```

```
 * Move the last element to the root.
```

```
*/
```

```
lastNode = getNodeAt(heap, heap->size - 1);
```

```
rootNode->email = lastNode->email;
```

```

/*
    * Find the node immediately before lastNode and unlink it.
    */

previous = getNodeAt(heap, heap->size - 2);
previous->next = NULL;

free(lastNode);

heap->size--;

/* Restore max-heap property. */
heapifyDown(heap, 0);

return result;
}

/* -----
Return the highest-priority email without removing it.
----- */

Email *peekMax(MaxHeap *heap)
{
    if (heap->size == 0 || heap->head == NULL)
        return NULL;

```

```

    return heap->head->email;
}

/* -----
   Free all heap nodes and email objects.
   ----- */

void destroyHeap(MaxHeap *heap)
{
    Node *current;
    Node *next;

    current = heap->head;

    while (current != NULL) {
        next = current->next;

        free(current->email);
        free(current);

        current = next;
    }

    heap->head = NULL;

```

```

    heap->size = 0;
}

/* -----
   Trim leading and trailing whitespace.
   ----- */

char *trimWhitespace(char *str)
{
    char *end;

    while (isspace((unsigned char)*str))
        str++;

    if (*str == '\0')
        return str;

    end = str + strlen(str) - 1;

    while (end > str && isspace((unsigned char)*end))
        end--;

    end[1] = '\0';

    return str;
}

```

```
}
```

```
/* -----
```

Parse an EMAIL command.

Expected:

EMAIL category,subject,date

```
----- */
```

*Email *parseEmail(const char *line)*

```
{
```

char buffer[MAX_LINE];

*char *firstComma;*

*char *secondComma;*

*char *category;*

*char *subject;*

*char *date;*

*Email *email;*

strncpy(buffer, line, sizeof(buffer) - 1);

buffer[sizeof(buffer) - 1] = '\0';

```
/*
```

** Skip "EMAIL".*

```
*/
```

```
category = buffer + 5;
```

```
category = trimWhitespace(category);
```

```
firstComma = strchr(category, ',');
```

```
if (firstComma == NULL)
```

```
    return NULL;
```

```
*firstComma = '\0';
```

```
subject = firstComma + 1;
```

```
secondComma = strchr(subject, ',');
```

```
if (secondComma == NULL)
```

```
    return NULL;
```

```
*secondComma = '\0';
```

```
date = secondComma + 1;
```

```
category = trimWhitespace(category);
```

```
subject = trimWhitespace(subject);
```

```
date = trimWhitespace(date);
```

```
if (getCategoryPriority(category) == 0)
```

```
    return NULL;
```

```
if (*subject == '\0' || *date == '\0')
```

```
    return NULL;
```

```
email = malloc(sizeof(Email));
```

```
if (email == NULL) {
```

```
    fprintf(stderr, "Memory allocation failed.\n");
```

```
    exit(EXIT_FAILURE);
```

```
}
```

```
strncpy(email->category, category, MAX_CATEGORY - 1);
```

```
email->category[MAX_CATEGORY - 1] = '\0';
```

```
strncpy(email->subject, subject, MAX_SUBJECT - 1);
```

```
email->subject[MAX_SUBJECT - 1] = '\0';
```

```
strncpy(email->date, date, MAX_DATE - 1);
```

```
email->date[MAX_DATE - 1] = '\0';
```

```
email->categoryPriority = getCategoryPriority(email->category);
```

```
email->dateValue = convertDate(email->date);
```

```
email->heapIndex = -1;
```

```
return email;
```

```
}
```

```
/* -----
```

```
Process commands from the input file.
```

```
----- */
```

```
void processCommands(FILE *input, MaxHeap *heap)
```

```
{
```

```
char line[MAX_LINE];
```

```
while (fgets(line, sizeof(line), input) != NULL) {
```

```
char *command;
```

```
command = trimWhitespace(line);
```

```
/* Ignore blank lines. */
```

```
if (*command == '\0')
```

```
continue;
```

```
if (strncmp(command, "EMAIL ", 6) == 0) {
```

```
Email *email = parseEmail(command);
```

```

    if (email != NULL) {
        insertHeap(heap, email);
    }
}

else if (strcmp(command, "NEXT") == 0) {
    Email *email = peekMax(heap);

    if (email != NULL) {
        printf("Next email:\n");
        printf("Sender: %s\n", email->category);
        printf("Subject: %s\n", email->subject);
        printf("Date: %s\n", email->date);
    }
}

else if (strcmp(command, "READ") == 0) {
    /*
     * removeMax() silently removes the current
     * highest-priority email.
     *
     * This also handles consecutive READ commands:
     * each READ removes one email.
     */
    Email *email = removeMax(heap);

```

```

        if (email != NULL)

            free(email);

    }

    else if (strcmp(command, "COUNT") == 0) {

        printf("There are %d emails to read.\n", heap->size);

    }

}

}

```

```

/* -----

```

Main

```

----- */

```

```

int main(int argc, char *argv[])

```

```

{

```

```

    FILE *input;

```

```

    MaxHeap heap;

```

```

/*

```

```

    * Usage:

```

```

    *   ./email_queue commands.txt

```

```

    */

```

```

if (argc != 2) {

```

```

    fprintf(stderr, "Usage: %s <input-file>\n", argv[0]);

```

```

    return EXIT_FAILURE;

```

```
}
```

```
input = fopen(argv[1], "r");
```

```
if (input == NULL) {
```

```
    perror("Could not open input file");
```

```
    return EXIT_FAILURE;
```

```
}
```

```
initHeap(&heap);
```

```
processCommands(input, &heap);
```

```
fclose(input);
```

```
destroyHeap(&heap);
```

```
return EXIT_SUCCESS;
```

```
}
```

Copilots code:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>

//

// Priority Queue Email MaxHeap implemented using a linked list

//

// ----- Email Struct -----

typedef struct Email {
    char sender[50];
    char subject[200];
    char date[20]; // MM-DD-YYYY
    int categoryPriority;
    int datePriority; // YYYYMMDD as integer for easy comparison
} Email;

// ----- Heap Node (Linked List) -----

typedef struct Node {
    Email email;
    struct Node* next;
} Node;

// Head of linked list representing heap
```

```
Node* head = NULL;
```

```
int heapSize = 0;
```

```
// ----- Utility Functions -----
```

```
int category_to_priority(const char* cat) {
```

```
    if (strcmp(cat, "Boss") == 0) return 5;
```

```
    if (strcmp(cat, "Subordinate") == 0) return 4;
```

```
    if (strcmp(cat, "Peer") == 0) return 3;
```

```
    if (strcmp(cat, "ImportantPerson") == 0) return 2;
```

```
    return 1; // OtherPerson
```

```
}
```

```
int date_to_int(const char* date) {
```

```
    // Convert MM-DD-YYYY → YYYYMMDD integer
```

```
    int mm, dd, yyyy;
```

```
    sscanf(date, "%d-%d-%d", &mm, &dd, &yyyy);
```

```
    return yyyy * 10000 + mm * 100 + dd;
```

```
}
```

```
// Get node at index (0-based)
```

```
Node* getNode(int index) {
```

```
    Node* curr = head;
```

```
    for (int i = 0; i < index && curr != NULL; i++)
```

```
        curr = curr->next;

    return curr;
}
```

// Swap email contents between two nodes

```
void swapEmails(Node* a, Node* b) {

    Email temp = a->email;

    a->email = b->email;

    b->email = temp;

}
```

// Compare priority of two emails

```
int compareEmails(Email a, Email b) {

    if (a.categoryPriority > b.categoryPriority) return 1;

    if (a.categoryPriority < b.categoryPriority) return -1;

    // Same category → compare date

    if (a.datePriority > b.datePriority) return 1;

    if (a.datePriority < b.datePriority) return -1;

    return 0;

}
```

// ----- Heapify Up -----

```
void heapifyUp(int index) {
```

```
if (index == 0) return;
```

```
int parent = (index - 1) / 2;
```

```
Node* childNode = getNode(index);
```

```
Node* parentNode = getNode(parent);
```

```
if (compareEmails(childNode->email, parentNode->email) > 0) {
```

```
    swapEmails(childNode, parentNode);
```

```
    heapifyUp(parent);
```

```
}
```

```
}
```

```
// ----- Heapify Down -----
```

```
void heapifyDown(int index) {
```

```
    int left = index * 2 + 1;
```

```
    int right = index * 2 + 2;
```

```
    Node* curr = getNode(index);
```

```
    if (!curr) return;
```

```
    int largest = index;
```

```
    Node* largestNode = curr;
```

```
Node* leftNode = getNode(left);
```

```
Node* rightNode = getNode(right);
```

```
if (leftNode && compareEmails(leftNode->email, largestNode->email) > 0) {
```

```
    largest = left;
```

```
    largestNode = leftNode;
```

```
}
```

```
if (rightNode && compareEmails(rightNode->email, largestNode->email) > 0) {
```

```
    largest = right;
```

```
    largestNode = rightNode;
```

```
}
```

```
if (largest != index) {
```

```
    swapEmails(curr, largestNode);
```

```
    heapifyDown(largest);
```

```
}
```

```
}
```

```
// ----- Insert Email -----
```

```
void insertEmail(Email e) {
```

```
    Node* newNode = (Node*)malloc(sizeof(Node));
```

```
    newNode->email = e;
```

```
    newNode->next = NULL;
```

```

if (head == NULL) {
    head = newNode;
} else {
    Node* curr = head;
    while (curr->next != NULL)
        curr = curr->next;
    curr->next = newNode;
}

heapSize++;
heapifyUp(heapSize - 1);
}

// ----- Get Max (Next) -----

Node* getMaxNode() {
    return head; // root of heap
}

// ----- Remove Max (Read) -----

void removeMax() {
    if (heapSize == 0) return;

```

```
if (heapSize == 1) {  
    free(head);  
    head = NULL;  
    heapSize = 0;  
    return;  
}
```

```
Node* last = getNode(heapSize - 1);
```

```
// Swap root with last  
swapEmails(head, last);
```

```
// Remove last node  
Node* curr = head;  
while (curr->next != last)  
    curr = curr->next;  
curr->next = NULL;  
free(last);
```

```
heapSize--;  
heapifyDown(0);  
}
```

```
// ----- Command Processing -----
```

```
void processEmailCommand(char* line) {  
    // Format: EMAIL <sender>,<subject>,<date>  
    char* data = line + 6; // skip "EMAIL "  
    char sender[50], subject[200], date[20];  
  
    // Extract fields  
    char* p1 = strtok(data, ",");  
    char* p2 = strtok(NULL, ",");  
    char* p3 = strtok(NULL, ",");  
  
    strcpy(sender, p1);  
    strcpy(subject, p2);  
    strcpy(date, p3);  
  
    Email e;  
    strcpy(e.sender, sender);  
    strcpy(e.subject, subject);  
    strcpy(e.date, date);  
    e.categoryPriority = category_to_priority(sender);  
    e.datePriority = date_to_int(date);  
  
    insertEmail(e);  
}
```

```
}
```

```
// ----- Main -----
```

```
int main() {
```

```
    char filename[100];
```

```
    printf("Enter command file name: ");
```

```
    scanf("%s", filename);
```

```
    FILE* fp = fopen(filename, "r");
```

```
    if (!fp) {
```

```
        printf("Could not open file.\n");
```

```
        return 1;
```

```
    }
```

```
    char line[300];
```

```
    while (fgets(line, sizeof(line), fp)) {
```

```
        line[strcspn(line, "\n")] = 0; // remove newline
```

```
        if (strncmp(line, "EMAIL", 5) == 0) {
```

```
            processEmailCommand(line);
```

```
        }
```

```
        else if (strcmp(line, "NEXT") == 0) {
```

```

    if (heapSize == 0) {
        printf("No emails to read.\n");
    } else {
        Node* max = getMaxNode();

        printf("Next email:\n");

        printf("Sender: %s\n", max->email.sender);

        printf("Subject: %s\n", max->email.subject);

        printf("Date: %s\n", max->email.date);
    }
}

else if (strcmp(line, "READ") == 0) {
    if (heapSize == 0) {
        // silently ignore
    } else {
        removeMax();
    }
}

else if (strcmp(line, "COUNT") == 0) {
    printf("There are %d emails to read.\n", heapSize);
}
}

fclose(fp);

return 0;

```

}

Code Analysis:

Correctness: Both programs were tested against the assignment's sample test file. Both produced output that exactly matched the expected result:

There are 6 emails to read.

Next email:

Sender: Boss

Subject: Never Mind

Date: 01-03-2025

Next email:

Sender: Boss

Subject: Important

Date: 12-20-2024

There are 4 emails to read.

Selection and Justification

I selected ChatGPT's implementation. It takes the input filename as a command-line argument, matching how the grading environment runs the program, while Copilot's version requires interactive scanf input and risks failing under automated grading. ChatGPT's version is also better commented and uses safer string handling (strncpy vs. strcpy).

Improvements Made

Added prolog comments (program name, description, inputs/outputs, author, dates, source attribution) since the generated code lacked a formal prolog block.